



PIC-UPV
PERTE CHIP CHAIR

Lab Book 3.0 Circuits and Compact Models

Student: Soraya Siying Latorre García y Esteban Ortiz Vicente

Instructions

General

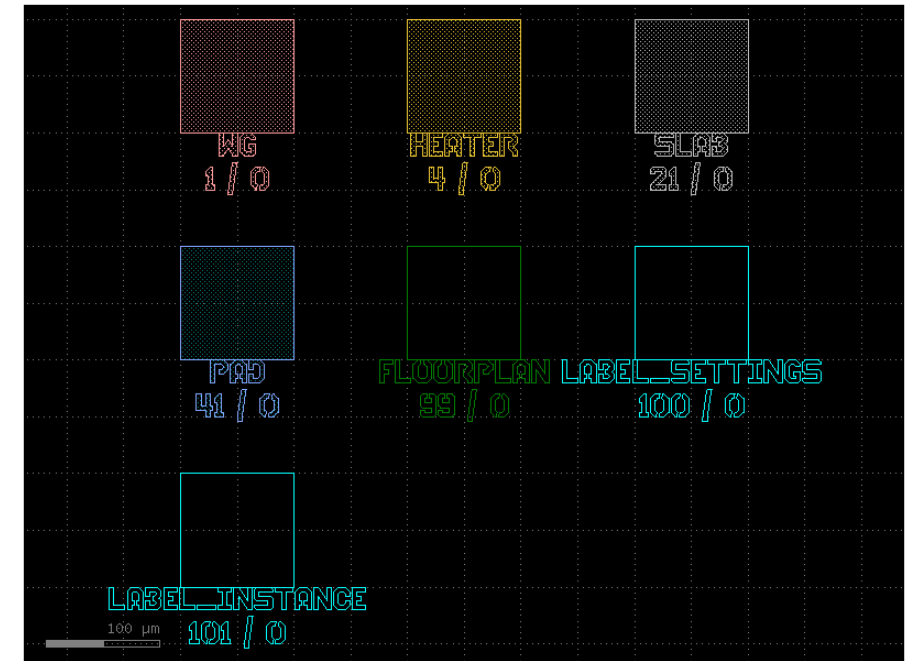
SCRIPTS

- From GitHub →
 - Fork the repo: <https://github.com/pascualmunoz/cifoin-lab4.git>
 - Clone the repo using VSCode
 - Open folder / repo in VSCode
 - Open terminal: uv sync
- Run the Jupyter Notebook:
 - LAYOUT_STUDENT.ipynb
- **Points**
 - **LO1: 3 points**
 - **LO2: 3 points**
 - **LO3: 1 points**
 - **LO4: 3 points**

Learning outcome #1a – Notebook section 1. Technology

“We will work with the demo UPVfab PDK. Let's view the layer distribution of this PDK”

```
LAYER_VIEWS = PDK.layer_views  
c = LAYER_VIEWS.preview_layerset()  
c.show()  
c.plot()
```



Un PDK (Process Design Kit) es una biblioteca de componentes fotónicos que describe y da acceso al proceso de fabricación de una foundry específica. Estos kits no solo contienen los componentes predefinidos, sino que también definen las diferentes capas del layout que representan los distintos pasos del proceso de fabricación. En general, cada capa corresponde a una máscara de litografía, que le indica a la foundry dónde aplicar cada paso del proceso: dónde grabar el silicio, dónde depositar metal, dónde añadir contactos, etc. De esta forma, el PDK actúa como el mediador entre el diseñador y la foundry, para que puedan entenderse fácilmente.

Learning outcome #1a – Notebook section 1. Technology

“We will work with the demo UPVfab PDK. Let's view the layer distribution of this PDK”

Para este PDK particular (UPVfab) encontramos las siguientes capas, con su número y descripción:

WG (1/0): Es la capa principal del diseño. Define las guías de onda de silicio por donde se propaga la luz. Todos los componentes ópticos se construyen sobre esta capa.

HEATER (4/0): Capa de calentador metálico. Permite modificar el índice de refracción del silicio mediante el efecto termo-óptico, lo que se usa para controlar la fase de la onda.

SLAB (21/0): Capa de silicio parcialmente grabado, que puede ser útil si se quiere reducir el espesor de silicio a los lados de la guía de onda, formando una guía tipo rib

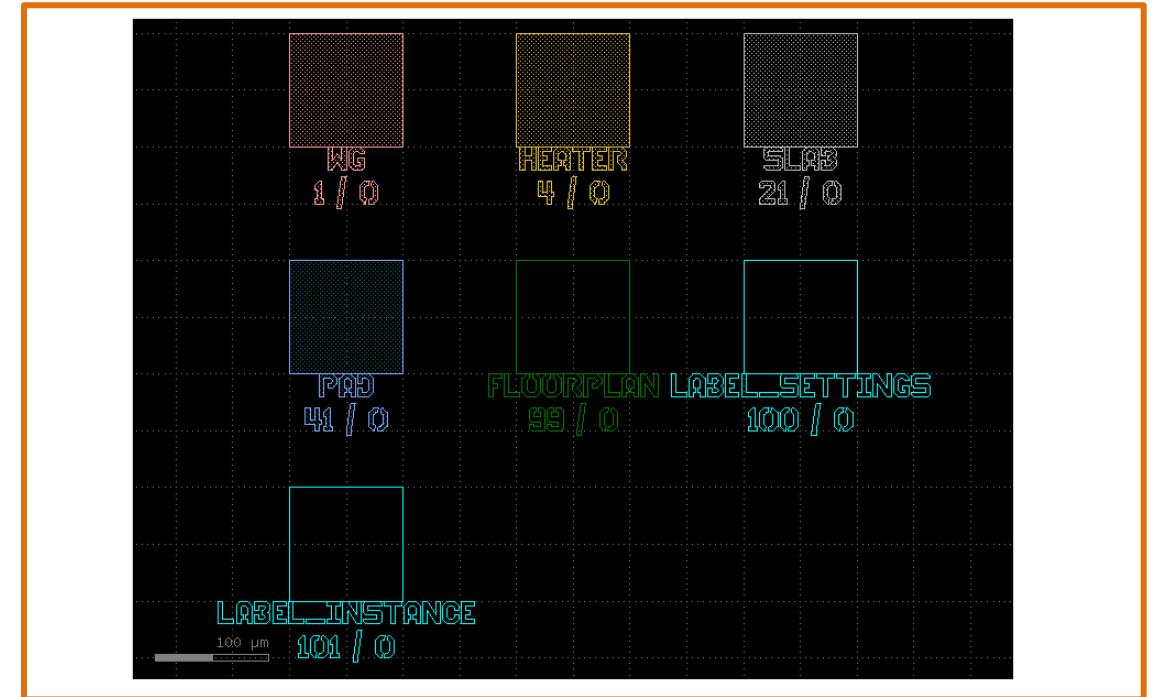
PAD (41/0): Capa que define las zonas de contacto eléctrico del chip

FLOORPLAN (99/0): Es la capa que define el área total del chip. Es el contorno exterior del diseño y le indica al foundry donde estará el chip integrado como tal.

LABEL_SETTINGS (100/0): Capa para configuración de etiquetas del diseño. Se usa para añadir información de texto sobre los componentes en el layout.

LABEL_INSTANCE (101/0): Capa para etiquetas de instancias específicas de componentes en el layout, útil para identificar cada elemento durante la verificación y fabricación.

Además, se observa que cada capa tiene asociados dos números. El primero de ellos indica la capa asociada a un material o paso del proceso de fabricación. El segundo número (datatipo) permite definir variantes dentro de una misma capa. En este PDK todas las capas tienen datatipo 0, lo que indica que se ha optado por asignar un número de capa distinto para cada material o uso



Learning outcome #1b – Notebook section 1. Technology – DC's

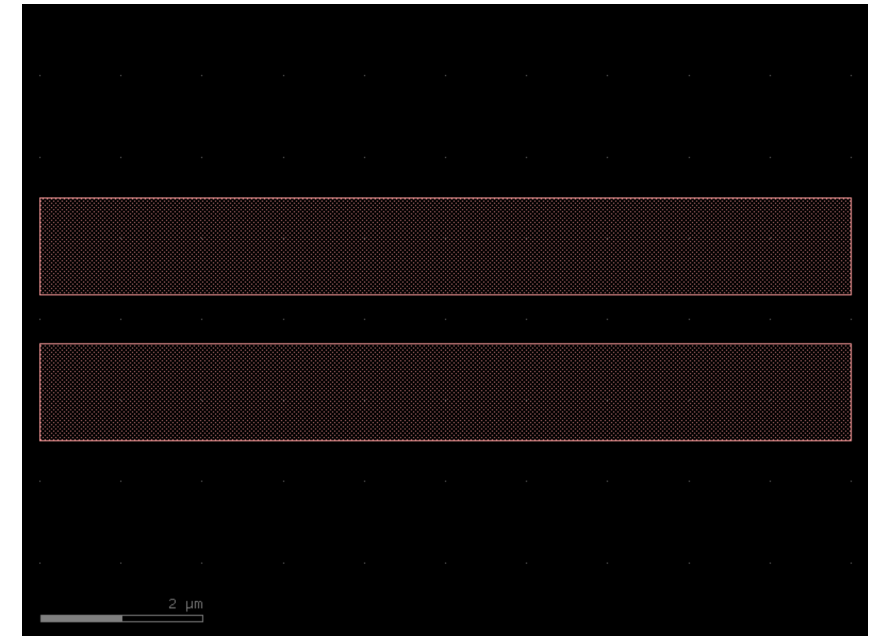
```
# Parámetros
core_width = 1.2 # μm
gap = 0.6 # μm
coupler_length = 10 # μm
separation = (core_width + gap) / 2 # 0.9 μm entre centros

# Layout
xs = gf.cross_section.strip(width=core_width, layer='WG')

c = gf.Component(name='DirectionalCoupler')

c.add_ref(gf.components.straight(length=coupler_length, cross_section=xs).move((0, separation)))
c.add_ref(gf.components.straight(length=coupler_length, cross_section=xs).move((0, -separation)))

c.plot()
```



Este es el acoplador direccional (DC) que diseñamos en lab 2 para la longitud de onda $\lambda = 1.55 \mu\text{m}$. Es un acoplador de guías de onda de 2 canales con acoplamiento evanescente, con el objetivo de conseguir una razón de acoplamiento de potencia definida por la separación entre guías. Las dimensiones relevantes son $\text{core_width} = 1.2 \mu\text{m}$, $\text{coupler_length} = 10 \mu\text{m}$, y $\text{gap} = 0.6 \mu\text{m}$ entre las guías. Las posiciones de entrada/salida con respecto al centro se encuentran en $\pm 0.9 \mu\text{m}$ en la dirección y.

Learning outcome #1b – Notebook section 1. Technology – MMI

```
# Parámetros del MMI
mmi_body_w = 12 # μm - Ancho del cuerpo MMI
mmi_length = 15 # μm - Longitud del MMI
access_width = 0.5 # μm - Ancho de las guías de acceso
access_length = 2 # μm - Longitud de las guías de acceso

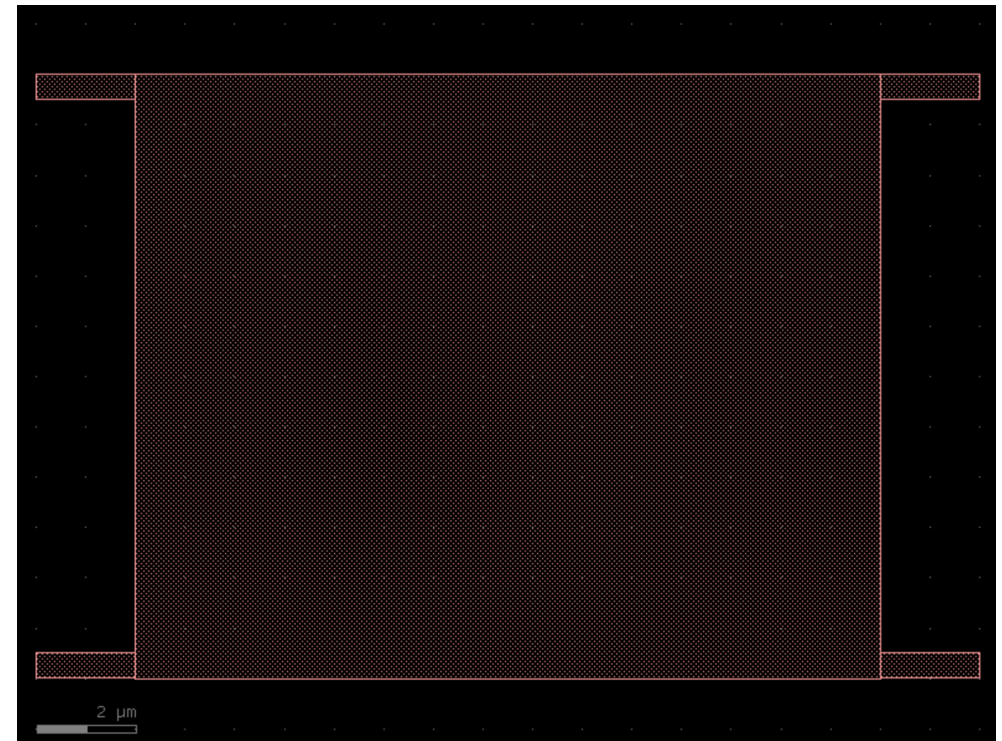
# Layout del MMI 2x2
c = gf.Component()

# Sección de acceso - Entradas
xs_access = gf.cross_section.strip(width=access_width, layer='WG')
c.add_ref(gf.components.straight(length=access_length, cross_section=xs_access).move((0, mmi_body_w/2 - access_width/2)))
c.add_ref(gf.components.straight(length=access_length, cross_section=xs_access).move((0, -mmi_body_w/2 + access_width/2)))

# Cuerpo MMI
xs_mmi = gf.cross_section.strip(width=mmi_body_w, layer='WG')
c.add_ref(gf.components.straight(length=mmi_length, cross_section=xs_mmi).move((access_length, 0)))

# Secciones de acceso - Salidas
y_out_pos = mmi_body_w/2 - access_width/2
c.add_ref(gf.components.straight(length=access_length, cross_section=xs_access).move((access_length + mmi_length, y_out_pos)))
c.add_ref(gf.components.straight(length=access_length, cross_section=xs_access).move((access_length + mmi_length, -y_out_pos)))

c.plot()
```



Este es el interferómetro multimodo (MMI) 2x2. Es un acoplador de interferencia con 2 entradas y 2 salidas, que proporciona una división de potencia equilibrada mediante la interferencia constructiva y destructiva de múltiples modos. Las dimensiones relevantes son $\text{mmi_body_w} = 12 \mu\text{m}$, $\text{mmi_length} = 15 \mu\text{m}$, $\text{access_width} = 0.5 \mu\text{m}$, y $\text{access_length} = 2 \mu\text{m}$. Las posiciones de entrada/salida con respecto al centro se encuentran en $\pm 5.75 \mu\text{m}$ en la dirección y .

Learning outcome #1b – Notebook section 1. Technology – MZIs

```
# Parámetros MMI
mmi_body_w = 12
mmi_length = 15
access_width = 0.5
access_length = 2
arm_length = 50

xs_access = gf.cross_section.strip(width=access_width, layer='WG')
xs_mmi = gf.cross_section.strip(width=mmi_body_w, layer='WG')

y_out_pos = mmi_body_w/2 - access_width/2
mmi_total = access_length + mmi_length + access_length

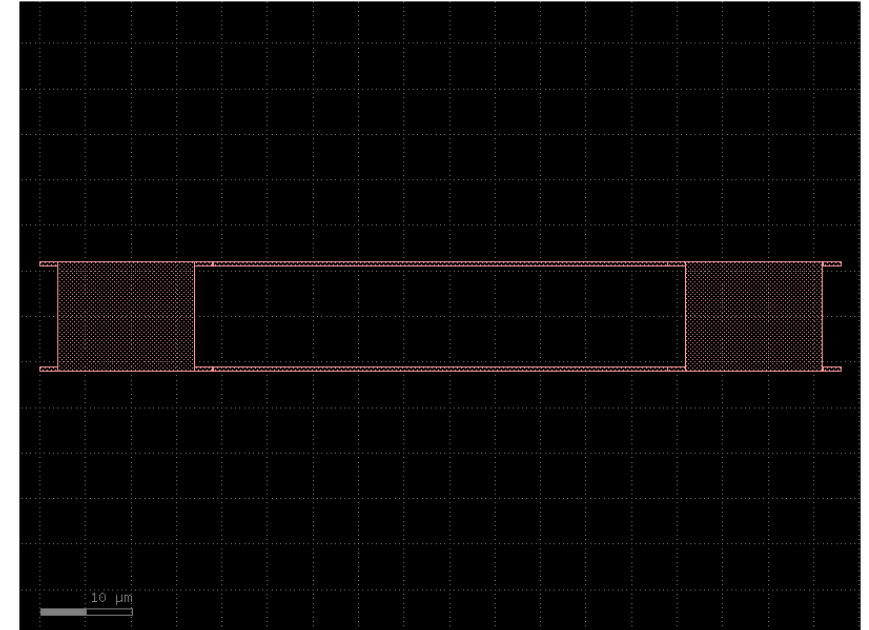
mzi = gf.Component()

mzi.add_ref(gf.components.straight(length=access_length, cross_section=xs_access)).move((0, y_out_pos))
mzi.add_ref(gf.components.straight(length=access_length, cross_section=xs_access)).move((0, -y_out_pos))
mzi.add_ref(gf.components.straight(length=mmi_length, cross_section=xs_mmi)).move((access_length, 0))
mzi.add_ref(gf.components.straight(length=access_length, cross_section=xs_access)).move((access_length + mmi_length, y_out_pos))
mzi.add_ref(gf.components.straight(length=access_length, cross_section=xs_access)).move((access_length + mmi_length, -y_out_pos))

mzi.add_ref(gf.components.straight(length=arm_length, cross_section=xs_access)).move((mmi_total, y_out_pos))
mzi.add_ref(gf.components.straight(length=arm_length, cross_section=xs_access)).move((mmi_total, -y_out_pos))

mzi.add_ref(gf.components.straight(length=access_length, cross_section=xs_access)).move((x2, y_out_pos))
mzi.add_ref(gf.components.straight(length=access_length, cross_section=xs_access)).move((x2, -y_out_pos))
mzi.add_ref(gf.components.straight(length=mmi_length, cross_section=xs_mmi)).move((x2 + access_length, 0))
mzi.add_ref(gf.components.straight(length=access_length, cross_section=xs_access)).move((x2 + access_length + mmi_length, y_out_pos))
mzi.add_ref(gf.components.straight(length=access_length, cross_section=xs_access)).move((x2 + access_length + mmi_length, -y_out_pos))

mzi.plot()
```



Se ha diseñado un interferómetro Mach-Zehnder (MZI) compuesto por dos acopladores MMI 1x2 conectados por dos brazos de longitud igual ($\text{arm_length} = 50 \mu\text{m}$). Cada MMI se ha construido manualmente combinando una sección central ancha ($\text{mmi_body_w} = 12 \mu\text{m}$) con guías de acceso estrechas ($\text{access_width} = 0.5 \mu\text{m}$) a la entrada y salida. La luz se divide en el primer MMI, recorre los dos brazos en paralelo y se recombina en el segundo MMI, produciendo una interferencia a la salida.

Learning outcome #1b – Notebook section 1. Technology – Ring Resonator

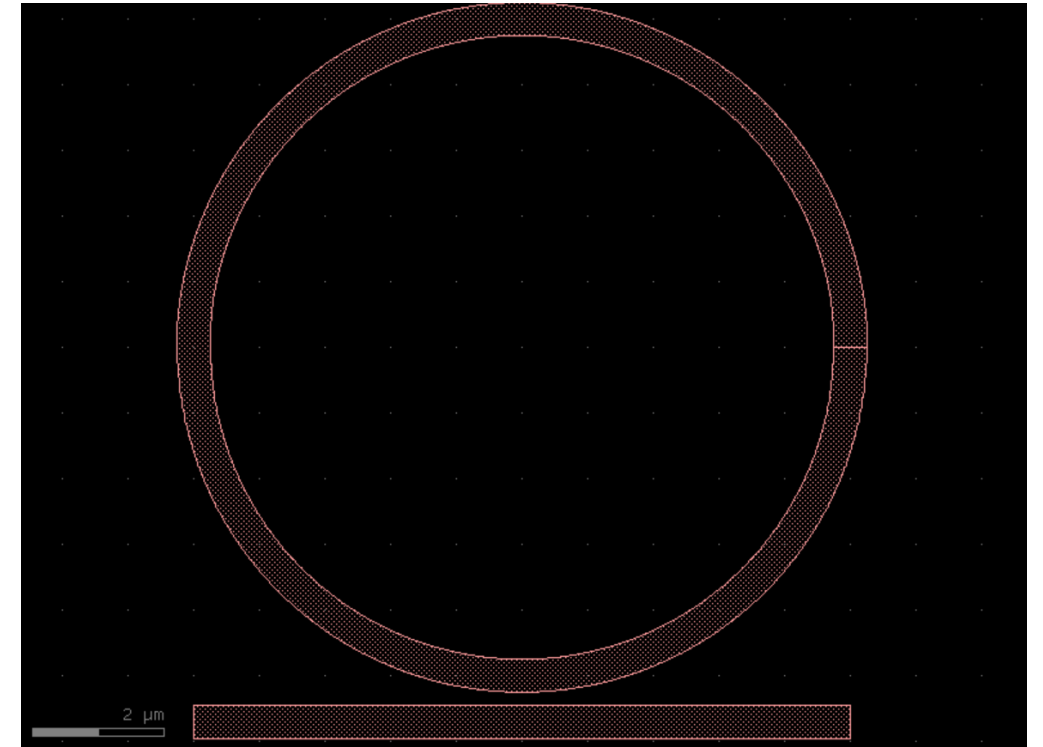
```
# Ring resonator |
c = gf.Component()

ring_radius = 5 # μm
waveguide_width = 0.5 # μm
gap = 0.2 * 3.5 # μm - Distancia entre anillo y guía acoplada

xs = gf.cross_section.strip(width=waveguide_width, layer='WG')

# Anillo (arco circular)
ring = gf.components.ring(radius=ring_radius, width=waveguide_width, layer='WG')
c.add_ref(ring).move((0, 0))

# Bus
bus = gf.components.straight(length=2*ring_radius, cross_section=xs)
c.add_ref(bus).move((-ring_radius, -(ring_radius+gap)))
```



Se ha diseñado un resonador de anillo (ring resonator) en configuración all-pass utilizando gdsfactory. El componente está formado por dos elementos: un anillo circular creado con `gf.components.ring` de radio 5 μm y anchura 0.5 μm, situado en el origen, y una guía de onda bus recta creada con `gf.components.straight` de longitud $2 \times \text{ring_radius} = 10 \mu\text{m}$ y misma anchura. El bus se ha posicionado debajo del anillo con un gap de 0.7 μm entre ambos, que controla el acoplamiento evanescente entre la guía de onda y el anillo.

Learning outcome #2a – Notebook section 2. Layout Fundamentals

“Once that we have a working 'new' component, we shall convert it into a Cell. This will allow us to have a hierarchical design”

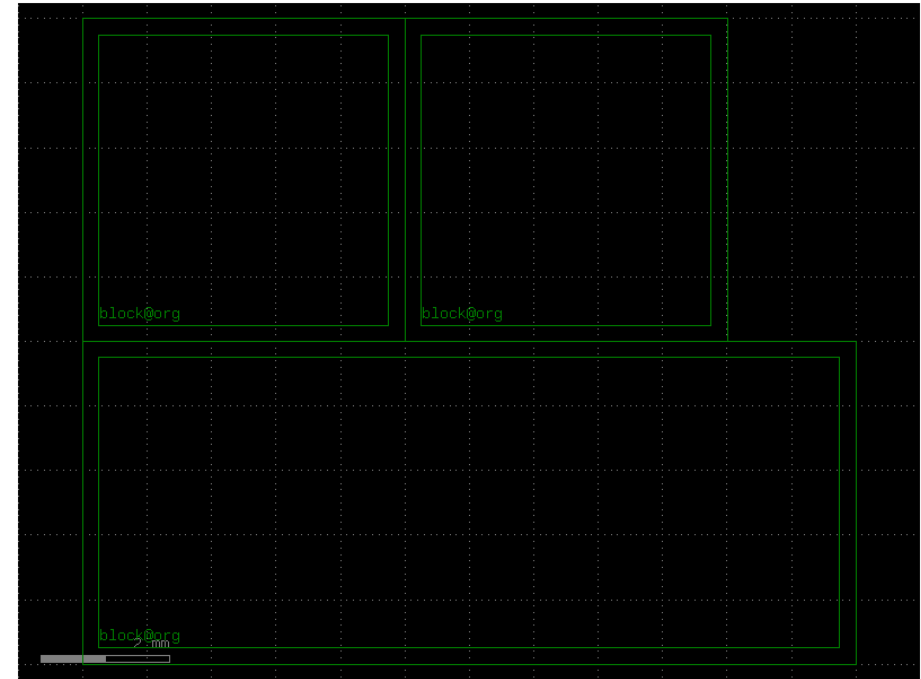
```
# 1. We define the cell as a function with it's corresponding parameters (and defaults)
gf.clear_cache()

@gf.cell
def die(diel = 5000, dielw = 5000, border = 250, layer_box = "FLOORPLAN"):
    # Die specifications (Chip)
    box = gf.Component()
    obox = box.add_ref(gf.components.rectangle(size=(dielw,dielw),layer=layer_box))
    ibox = box.add_ref(gf.components.rectangle(size=(dielw-border*2,dielw-border*2),layer=layer_box)).dmovex(border).dmovex(border)
    box = gf.boolean(A=obox, B=ibox, operation="A-B", layer=layer_box)
    # Adding ports to a component
    box.add_port(name="block@org", center=[border,border], width=1, orientation=0, layer=layer_box)
    box.draw_ports()
    return box

# 2. We instantiate references of each cell. We can also name it individually to avoid problems
wafer = gf.Component()
dielw1 = 5000
dielw2 = 10000

c1 = wafer.add_ref(die(dielw = dielw1))
c2 = wafer.add_ref(die(dielw = dielw1))
c3 = wafer.add_ref(die(dielw = 12000))

c2.dmovex(dielw1)
c3.dmovey(-5000)
wafer.show()
wafer.plot()
```



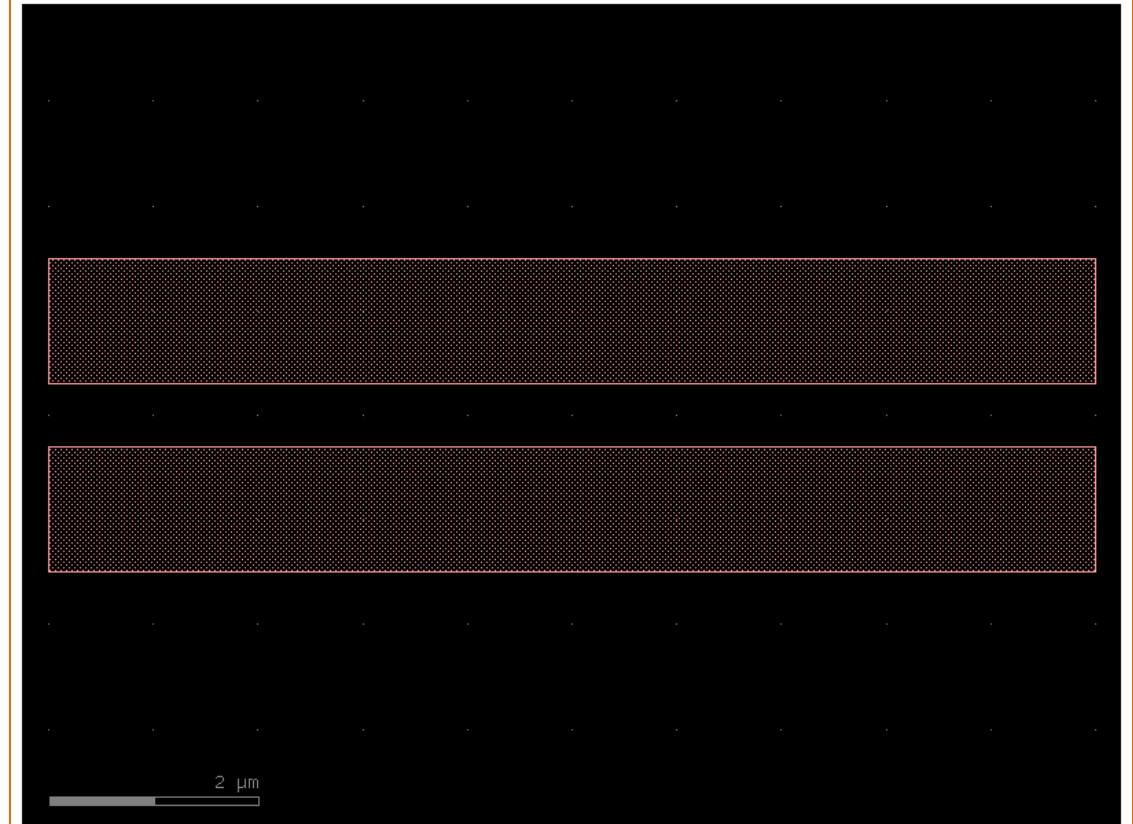
Se muestra una celda die que representa el contorno del chip usando la capa FLOORPLAN. Se construye restando dos rectángulos para generar un marco con un borde de 250 μm . Se instancian tres dies de distintos tamaños en un wafer común, lo que permite organizar y visualizar cómo quedarían distribuidos los chips en el wafer antes de la fabricación. Aquí se muestra la base, a partir de aquí se irán colocando todos los componentes fotónicos que se usarán en el diseño del chip.

Learning outcome #2a – Notebook section 2. Layout Fundamentals – DCs

```
# Learning outcome #2a - Notebook section 2. Layout Fundamentals - DCs

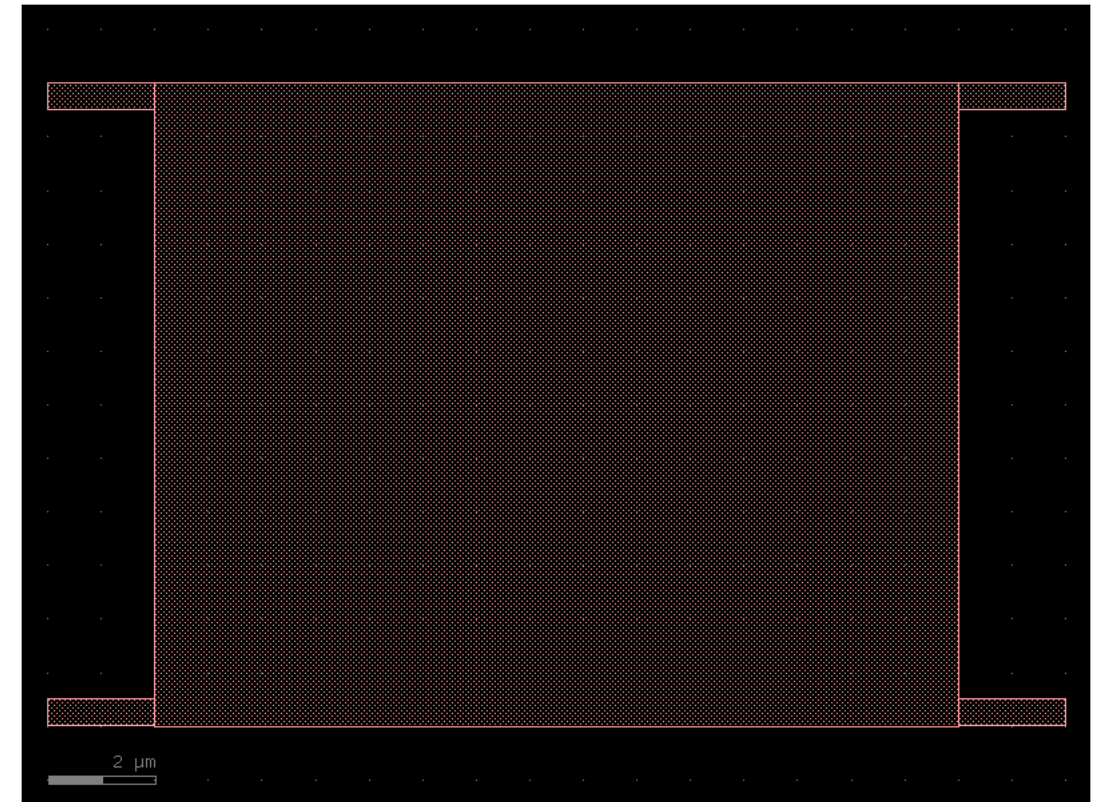
@gf.cell
def dc_coupler(core_width=1.2, gap=0.6, coupler_length=10):
    separation = (core_width + gap) / 2; xs = gf.cross_section.strip(width=core_width, layer='WG'); c = gf.Component()
    c.add_ref(gf.components.straight(length=coupler_length, cross_section=xs).move((0, separation)))
    c.add_ref(gf.components.straight(length=coupler_length, cross_section=xs).move((0, -separation)))
    return c

c = gf.Component(); c.add_ref(dc_coupler()).move((0, 0)); c.plot()
```



Learning outcome #2a – Notebook section 2. Layout Fundamentals – MMIs

```
# Learning outcome #2a - Notebook section 2. Layout Fundamentals - MMIs#  
  
@gf.cell  
def mmi(mmi_body_w=12, mmi_length=15, access_width=0.5, access_length=2):  
  
    xs_access = gf.cross_section.strip(width=access_width, layer='WG')  
    xs_mmi = gf.cross_section.strip(width=mmi_body_w, layer='WG')  
    y = mmi_body_w/2 - access_width/2  
  
    c = gf.Component()  
  
    c.add_ref(gf.components.straight(length=access_length, cross_section=xs_access)).move((0, y))  
    c.add_ref(gf.components.straight(length=access_length, cross_section=xs_access)).move((0, -y))  
  
    c.add_ref(gf.components.straight(length=mmi_length, cross_section=xs_mmi)).move((access_length, 0))  
  
    c.add_ref(gf.components.straight(length=access_length, cross_section=xs_access)).move((access_length + mmi_length, y))  
    c.add_ref(gf.components.straight(length=access_length, cross_section=xs_access)).move((access_length + mmi_length, -y))  
  
    return c  
  
c = gf.Component(); c.add_ref(mmi()).move((0,0)); c.plot()
```



Learning outcome #2a – Notebook section 2. Layout Fundamentals – MZIs

```
@gf.cell
def mzi(mmi_body_w=12, mmi_length=15, access_width=0.5, access_length=2, arm_length=50):

    xs_access = gf.cross_section.strip(width=access_width, layer='WG')
    xs_mmi = gf.cross_section.strip(width=mmi_body_w, layer='WG')

    y = mmi_body_w/2 - access_width/2
    mmi_total = access_length + mmi_length + access_length
    x2 = mmi_total + arm_length

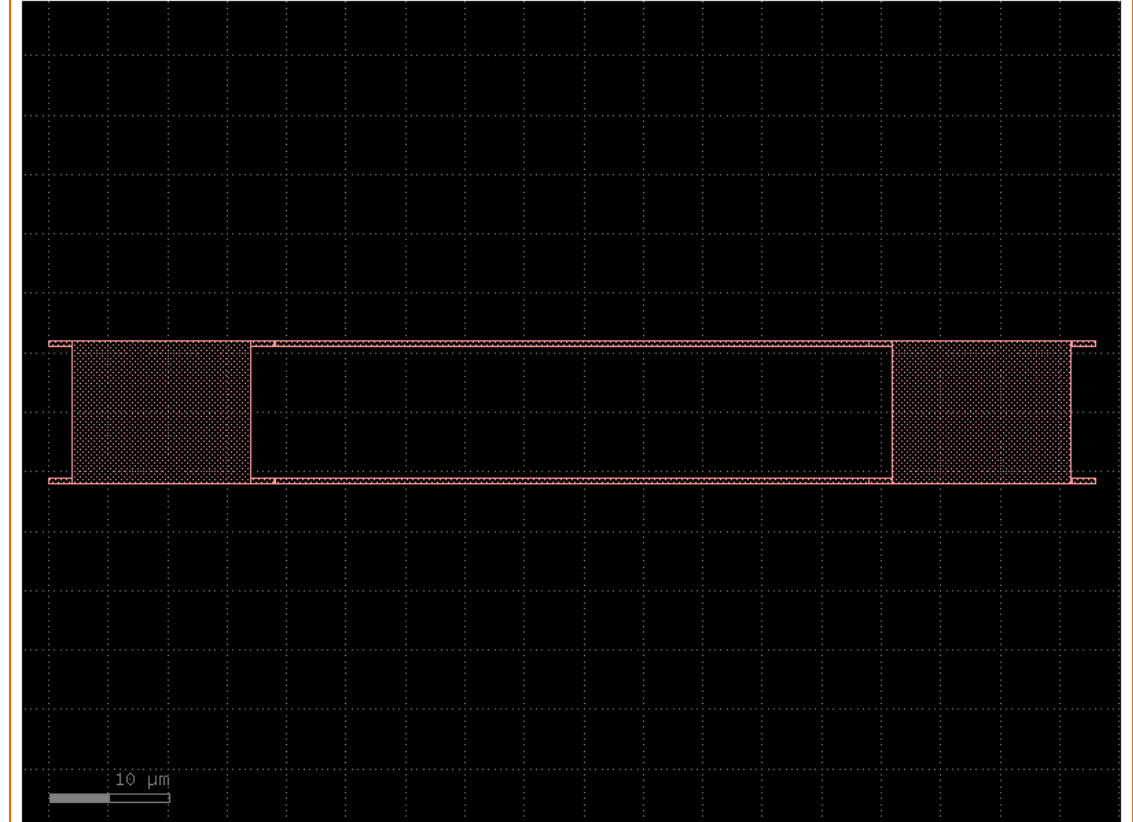
    c = gf.Component()

    c.add_ref(gf.components.straight(length=access_length, cross_section=xs_access)).move((0, y))
    c.add_ref(gf.components.straight(length=access_length, cross_section=xs_access)).move((0, -y))
    c.add_ref(gf.components.straight(length=mmi_length, cross_section=xs_mmi)).move((access_length, 0))
    c.add_ref(gf.components.straight(length=access_length, cross_section=xs_access)).move((access_length + mmi_length, y))
    c.add_ref(gf.components.straight(length=access_length, cross_section=xs_access)).move((access_length + mmi_length, -y))
    c.add_ref(gf.components.straight(length=arm_length, cross_section=xs_access)).move((mmi_total, y))
    c.add_ref(gf.components.straight(length=arm_length, cross_section=xs_access)).move((mmi_total, -y))

    c.add_ref(gf.components.straight(length=access_length, cross_section=xs_access)).move((x2, y))
    c.add_ref(gf.components.straight(length=access_length, cross_section=xs_access)).move((x2, -y))
    c.add_ref(gf.components.straight(length=mmi_length, cross_section=xs_mmi)).move((x2 + access_length, 0))
    c.add_ref(gf.components.straight(length=access_length, cross_section=xs_access)).move((x2 + access_length + mmi_length, y))
    c.add_ref(gf.components.straight(length=access_length, cross_section=xs_access)).move((x2 + access_length + mmi_length, -y))

    return c

c = gf.Component(); c.add_ref(mzi()).move((0,0)); c.plot()
```



Learning outcome #2a – Notebook section 2. Layout Fundamentals – Ring Resonator

```
@gf.cell
def ring_resonator(ring_radius=5, waveguide_width=0.5, gap=0.5):

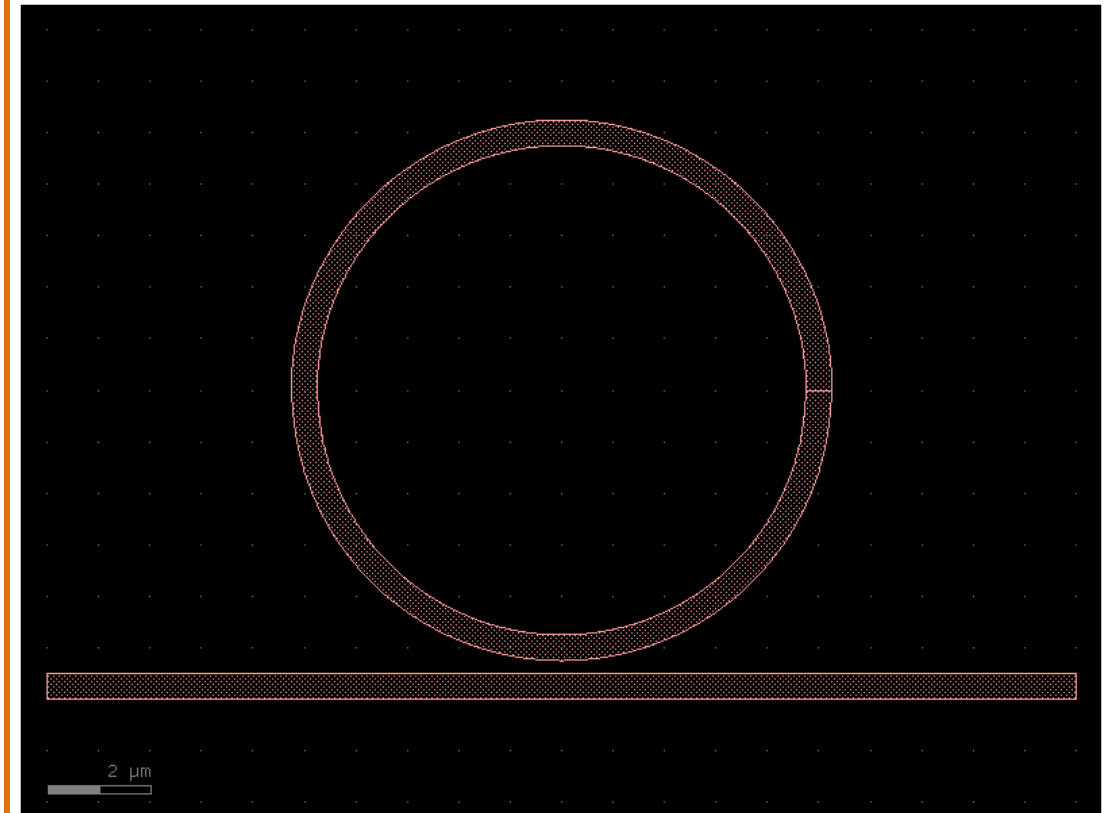
    c = gf.Component()
    xs = gf.cross_section.strip(width=waveguide_width, layer='WG')

    # Anillo centrado en el origen
    ring = gf.components.ring(radius=ring_radius, width=waveguide_width, layer='WG')
    ring_ref = c.add_ref(ring)
    ring_ref.move((0, ring_radius)) # lo subimos para que el bus quede debajo

    # Bus centrado bajo el anillo
    bus_length = 4 * ring_radius
    bus = gf.components.straight(length=bus_length, cross_section=xs)
    bus_ref = c.add_ref(bus)
    bus_ref.move((-bus_length / 2, -(waveguide_width / 2 + gap)))

    return c

c = ring_resonator()
c.plot()
```



Learning outcome #3 – Notebook section 3. Complete layout

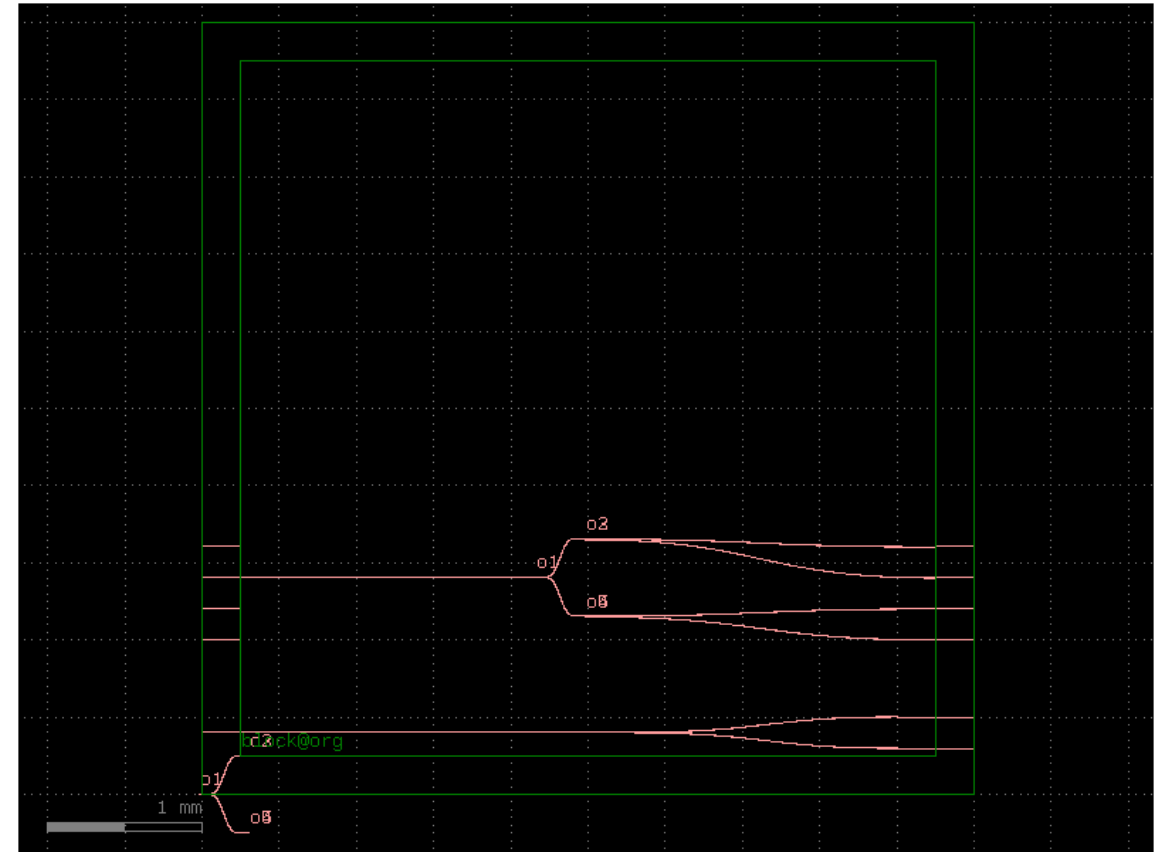
Read and describe 4 code blocks in section 3

Python code block 1: Este código usa GDSFactory para diseñar un chip fotónico con un divisor de haz MMI 1×2. Se define un die de 5×5 mm y se coloca un MMI cerca del borde izquierdo, que divide una señal óptica en dos salidas. Se añaden tres waveguides rectos de 500 μm en los bordes del chip para garantizar entradas y salidas perpendiculares, y se conectan al MMI mediante curvas en S (route_single_sbend), visibles como las líneas rosas en la imagen. Finalmente, el diseño se visualiza con main.plot() y main.show().

Python code block 2: Ahora se construye un árbol MMI 1×4. Se colocan tres divisores MMI 1×2: el primero (mmi1) actúa como raíz del árbol, y los otros dos (mmi2 y mmi3) se desplazan a la derecha separados verticalmente por pad_y = 250 μm . Las salidas o2 y o3 de mmi1 se conectan a las entradas de mmi2 y mmi3 mediante curvas en S, formando la estructura en árbol. Por último, se registran los 5 puertos del componente (1 entrada y 4 salidas) para que sean accesibles desde fuera del componente.

Python code block 3: Se instancia el árbol MMI 1×4 definido y se añade al componente principal main, que es el die?

Python code block 4: Se crean dos arrays de 4 waveguides rectos de 250 μm de longitud, con una separación de 200 μm entre ellos: in_arr posicionado en el borde izquierdo del die y out_arr en el borde derecho, ambos desplazados 1000 μm verticalmente. Estos actúan como los puertos de entrada y salida del chip., que actúan como los puertos de entrada y salida del chip en los bordes del die. Luego, se conecta la entrada del array al puerto o1 del árbol, y las 4 salidas del árbol (o2-o5) a los 4 waveguides del array de salida.



Learning outcome #4 – Notebook section 4. Exercises – 1 slide per bullet point

Code for ...

Part a) Creating components

- Create a cell component for a unbalanced MZI, using 2x2 50:50 MMIs with arm length difference as parameter
- Create a cell component based on the previous, where the arms have thermal tuners on top of each one
- Create a cell component for an all-pass ring resonator, using 2x2 50:50 MMIs, with extra length parameter for different perimeters
- Create a cell component based on the previous, where the ring has a thermal tuner on top along all the perimeter
- Create a cell component based on existing GDSfactory spiral components, with length as parameter

Part b) Creating die

- Create a die $W = 5 \text{ mm} \times L = 10 \text{ mm}$
- Create an array of I/Os spaced at $250 \text{ }\mu\text{m}$ (as much I/Os as the width allows)
- Create a cell component of this die, with I/Os accessible to connect

Part c) Floorplaning and die layout

- Make an instance of the die as host component for your layout
- Add 3 sets of 3 straight waveguides, from left to right of the die, top, middle and bottom of the die
- Add 2 of each of the components above, with different lengths
- Connect all your components to the I/Os

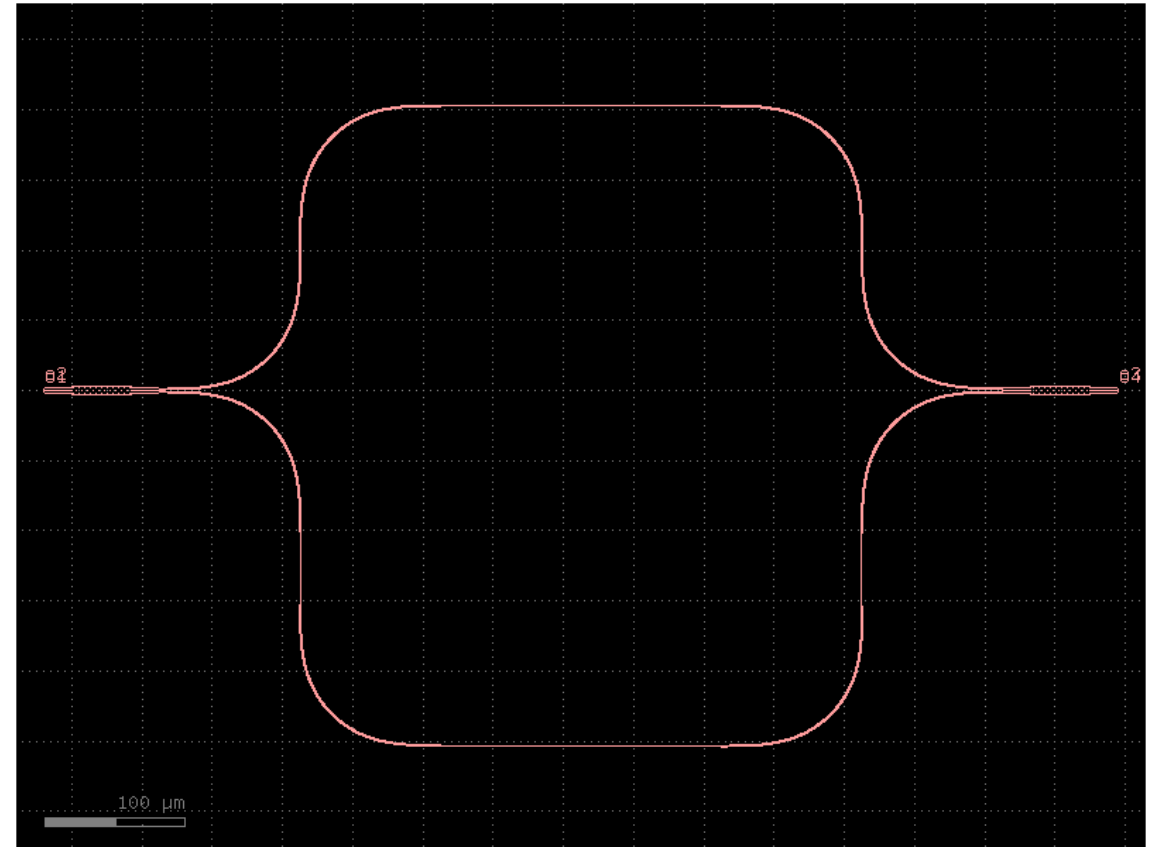
Screenshot for ...

- Create a cell component for a unbalanced MZI, using 2x2 50:50 MMIs with arm length difference as parameter

```
@gf.cell
def mzi_unbalanced_2x2(
    delta_length: float = 10.0,
    length_y: float = 2.0,
    length_x: float = 200.0,
) -> gf.Component:

    c = gf.components.mzis.mzi2x2_2x2_phase_shifter(
        delta_length=delta_length,
        length_y=length_y,
        length_x=length_x,
        bend="bend_euler",
        straight="straight",
        straight_x_top="straight",
        straight_x_bot="straight",
        splitter="mmi2x2",
        combiner="mmi2x2",
        with_splitter=True,
        port_e1_splitter="o3",
        port_e0_splitter="o4",
        port_e1_combiner="o3",
        port_e0_combiner="o4",
        port1="o1",
        port2="o2",
        nbends=2,
        cross_section="strip",
        mirror_bot=False,
        add_optical_ports_arms=False,
        min_length=0.01,
        auto_rename_ports=True,
        auto_detect_port_names=False,
    ).copy()

    return c
```



- Create a cell component based on the previous, where the arms have thermal tuners on top of each one

```
@gf.cell
def mzi_unbalanced_2x2_heaters(
    delta_length: float = 10.0,
    length_y: float = 2.0,
    length_x: float = 200.0,
) -> gf.Component:

    c = gf.components.mzis.mzi2x2_2x2_phase_shifter(
        delta_length=delta_length,
        length_y=length_y,
        length_x=length_x,
        bend="bend_euler",
        straight="straight",

        # Hemos usamos nuestra versión compatible
        straight_x_top=straight_heater_metal_compatible,
        straight_x_bot=straight_heater_metal_compatible,

        splitter="mmi2x2",
        combiner="mmi2x2",
        with_splitter=True,

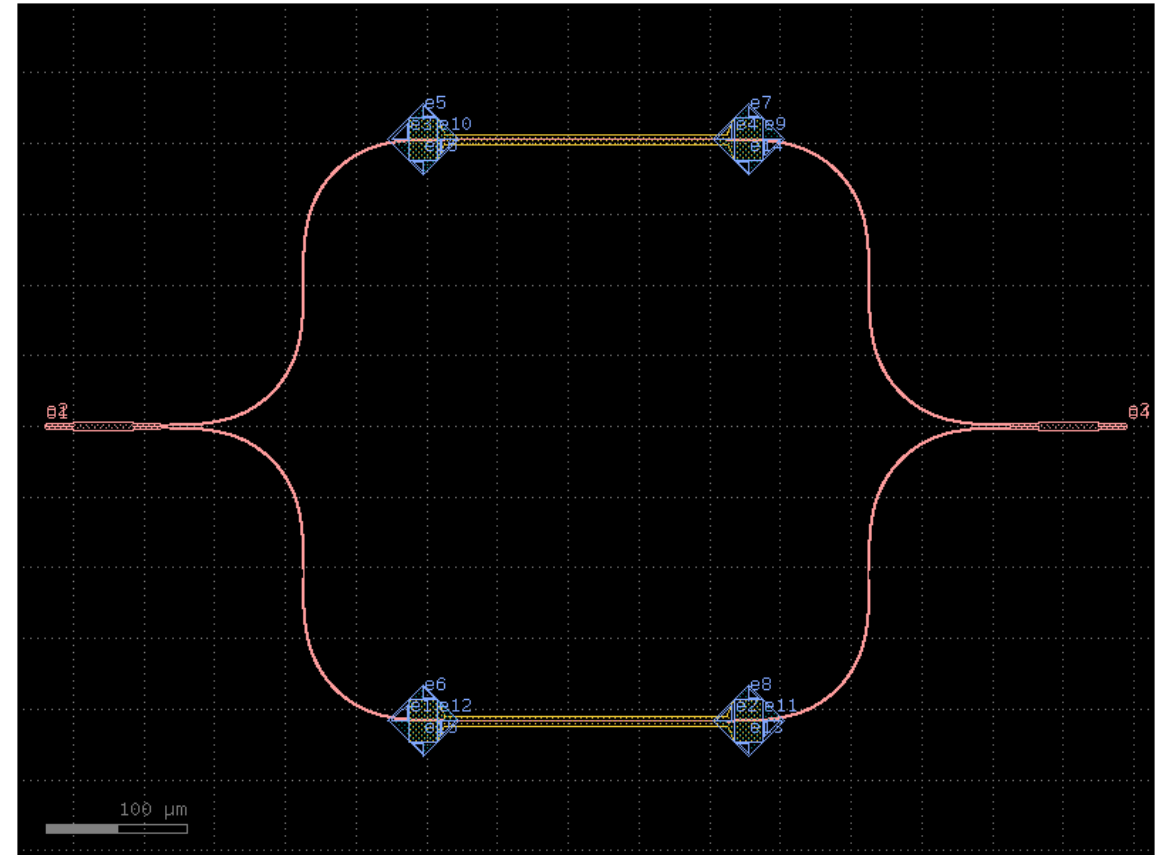
        port_e1_splitter="o3",
        port_e0_splitter="o4",
        port_e1_combiner="o3",
        port_e0_combiner="o4",

        port1="o1",
        port2="o2",
        nbends=2,
        cross_section="strip",
        mirror_bot=False,
        add_optical_ports_add_arms=False,
        min_length=0.01,
        auto_rename_ports=True,
        auto_detect_port_names=False,
    ).copy()
```

```
@gf.cell
def straight_heater_metal_compatible(
    length: float = 10.0,
    cross_section=None,
    **kwargs,
) -> gf.Component:

    c = gf.components.straight_heater_metal(
        length=length,
    )

    return c
```



- Create a cell component for an all-pass ring resonator, using 2x2 50:50 MMIs, with extra length parameter for different perimeters

```
@gf.cell
def all_pass_ring_mmi2x2(
    extra_length: float = 0.0,
    loop_height: float = 60.0,
    loop_width: float = 120.0,
) -> gf.Component:

    c = gf.Component()

    xs = gf.cross_section.strip(width=1.0, layer="WG")

    mmi = c.add_ref(cells.mmi2x2())

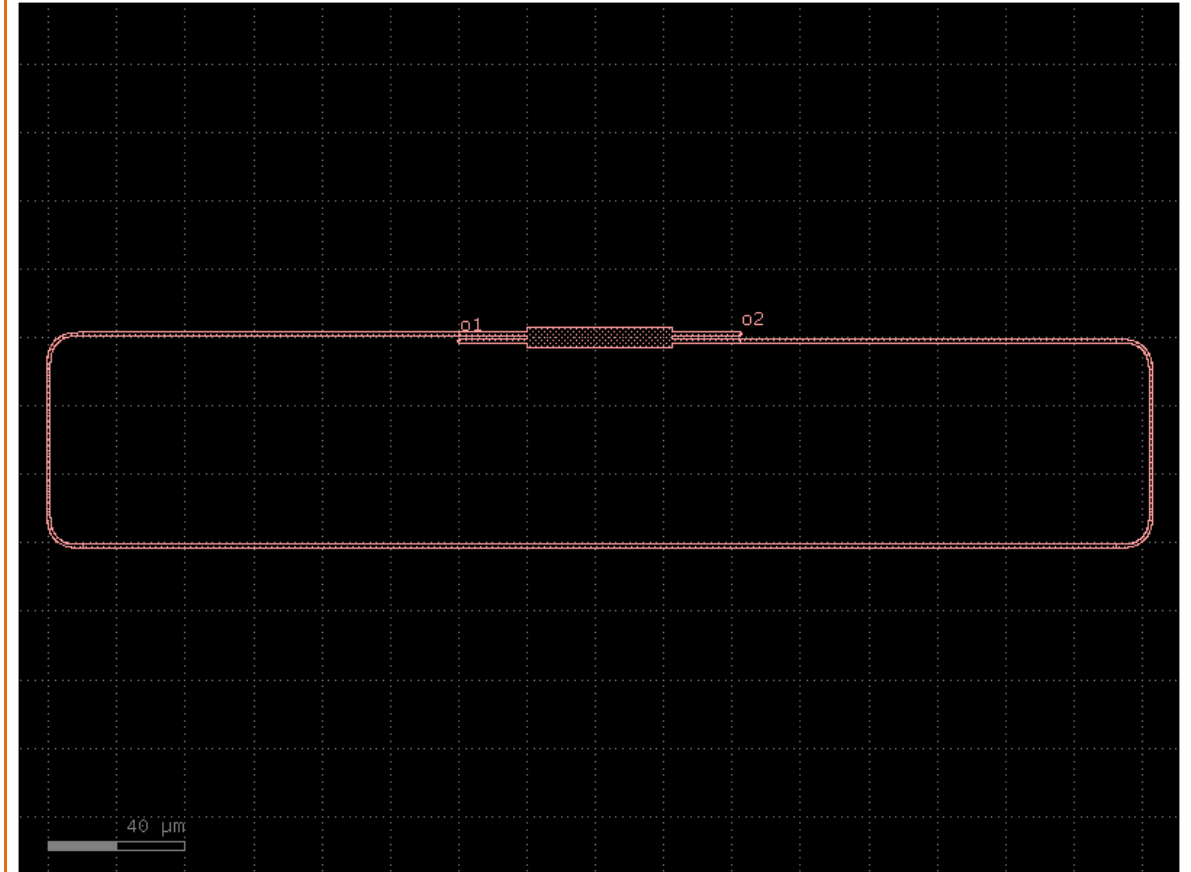
    p1 = mmi.ports["o4"]
    p2 = mmi.ports["o2"]
    x0 = p1.center[0]
    y0 = p1.center[1]
    x1 = p2.center[0]
    y1 = p2.center[1]

    y_loop = min(y0, y1) - loop_height - extra_length / 2

    gf.routing.route_single(
        c,
        port1=p1,
        port2=p2,
        waypoints=[
            (x0 + loop_width, y0),
            (x0 + loop_width, y_loop),
            (x1 - loop_width, y_loop),
            (x1 - loop_width, y1),
        ],
        cross_section=xs,
    )

    # Puertos externos del all-pass
    c.add_port("o1", port=mmi.ports["o1"]) # input
    c.add_port("o2", port=mmi.ports["o3"]) # output

    return c
```



- Create a cell component based on the previous, where the ring has a thermal tuner on top along all the perimeter

```
@gf.cell
def all_pass_ring_mmi2x2_heater(
    extra_length: float = 0.0,
    loop_height: float = 60.0,
    loop_width: float = 120.0,
    heater_width: float = 3.0,
) -> gf.Component:

    c = gf.Component()
    xs_wg = gf.cross_section.strip(width=1.0, layer="WG")

    mmi = c.add_ref(cells.mmi2x2())
    p_start, p_end = mmi.ports["o4"], mmi.ports["o2"]

    x0, y0 = p_start.center
    x1, y1 = p_end.center

    y_loop = min(y0, y1) - loop_height - extra_length / 2

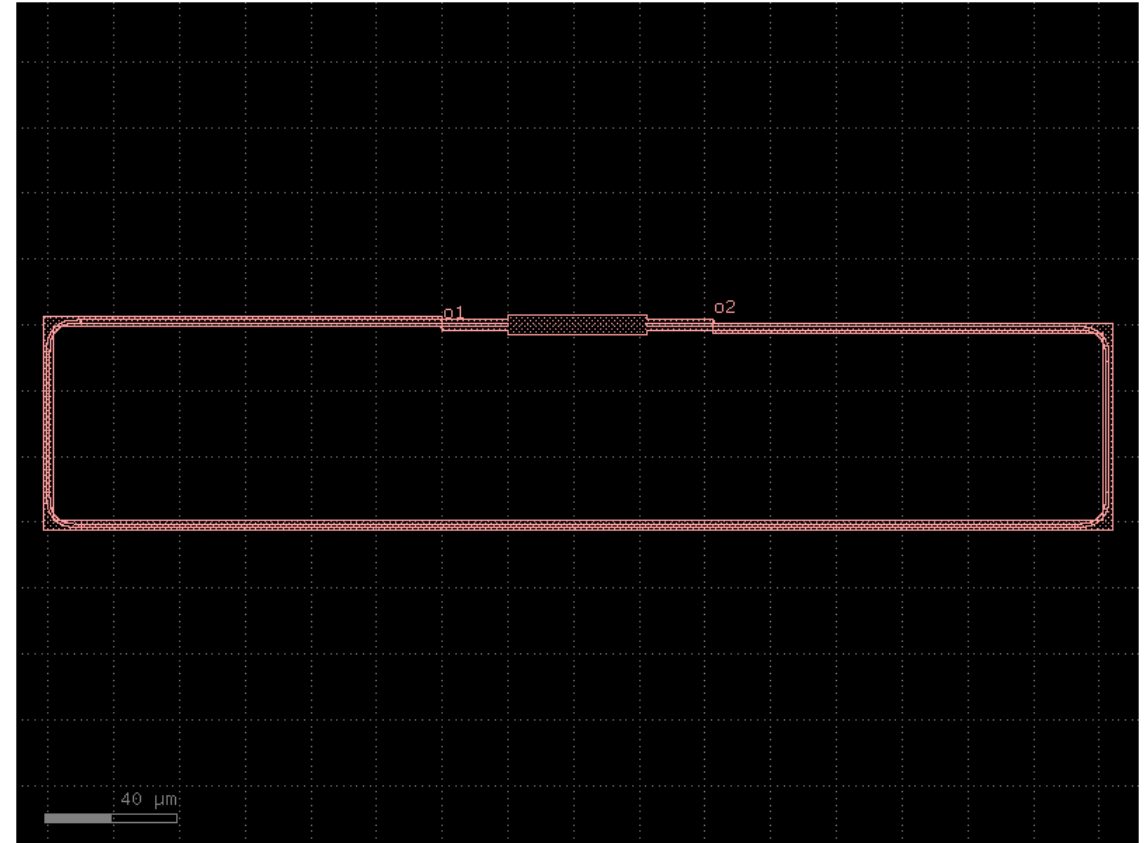
    points = [
        p_start.center,
        (x0 + loop_width, y0),
        (x0 + loop_width, y_loop),
        (x1 - loop_width, y_loop),
        (x1 - loop_width, y1),
        p_end.center,
    ]

    gf.routing.route_single(
        c,
        port1=p_start,
        port2=p_end,
        waypoints=points[1:-1],
        cross_section=xs_wg,
    )

    c.add_ref(
        gf.path.extrude(
            gf.Path(points),
            cross_section=gf.cross_section.strip(
                width=heater_width,
                layer="WG",
            ),
        ),
    )

    c.add_port("o1", port=mmi.ports["o1"])
    c.add_port("o2", port=mmi.ports["o3"])

    return c
```

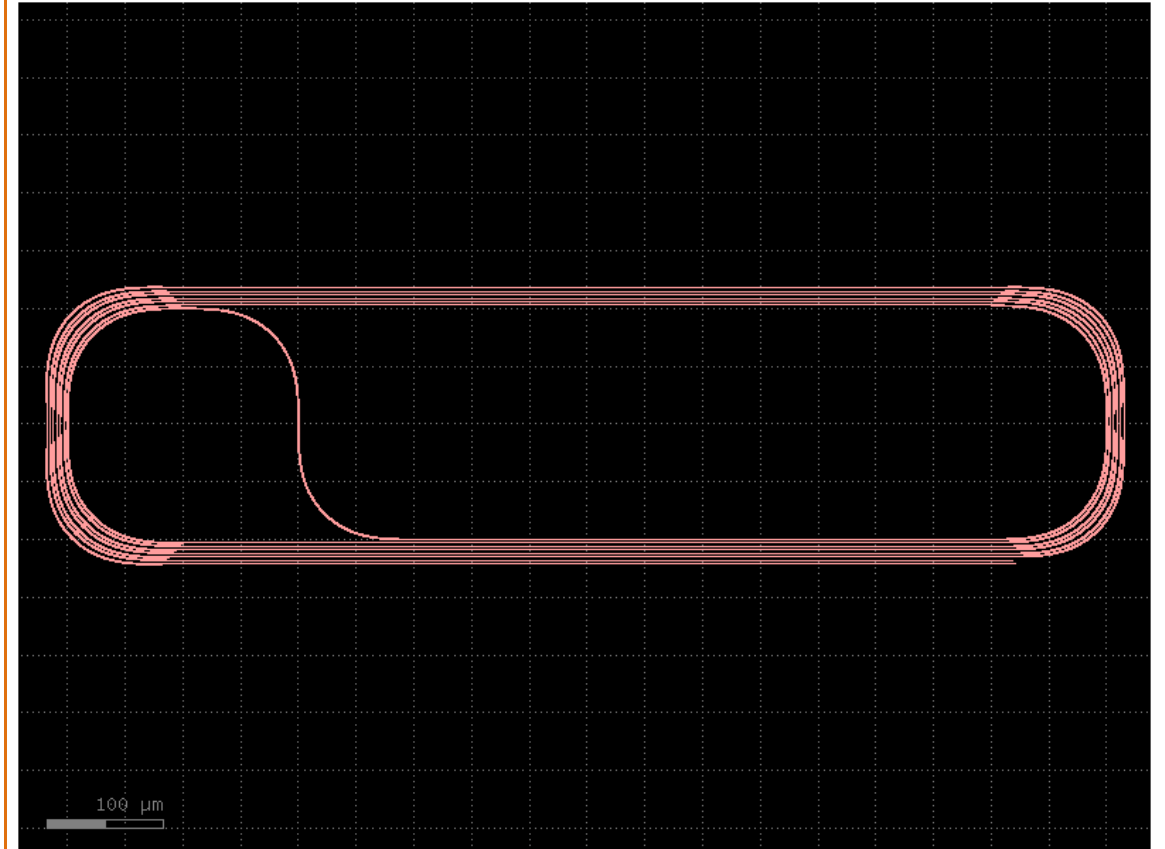


- Create a cell component based on existing GDSfactory spiral components, with length as parameter

```
@gf.cell
def spiral_length(
    length: float = 100.0,
    spacing: float = 3.0,
    n_loops: int = 6,
) -> gf.Component:

    c = gf.components.spiral(
        length=length,
        bend="bend_euler",
        straight="straight",
        cross_section="strip",
        spacing=spacing,
        n_loops=n_loops,
    ).copy()

    return c
```



Part b) Creating die - Create a die W =5 mm x L = 10 mm

```
@gf.cell
def die_5x10mm(
    width: float = 5000.0,
    length: float = 10000.0,
    border_width: float = 10.0,
) -> gf.Component:

    c = gf.Component()

    bottom = c.add_ref(gf.components.rectangle(size=(length, border_width), layer="WG"))
    bottom.move((0, 0))

    top = c.add_ref(gf.components.rectangle(size=(length, border_width), layer="WG"))
    top.move((0, width - border_width))

    left = c.add_ref(gf.components.rectangle(size=(border_width, width), layer="WG"))
    left.move((0, 0))

    right = c.add_ref(gf.components.rectangle(size=(border_width, width), layer="WG"))
    right.move((length - border_width, 0))

    return c
```



- Create an array of I/Os spaced at 250 μm (as much I/Os as the width allows)

```
@gf.cell
def die_cell(
    width: float = 5000.0,
    length: float = 10000.0,
    border_width: float = 10.0,
    io_pitch: float = 250.0,
    io_width: float = 1.0,
    io_length: float = 50.0,
) -> gf.Component:

    c = gf.Component()

    bottom = c.add_ref(gf.components.rectangle(size=(length, border_width), layer="WG"))
    bottom.move((0, 0))

    top = c.add_ref(gf.components.rectangle(size=(length, border_width), layer="WG"))
    top.move((0, width - border_width))

    left = c.add_ref(gf.components.rectangle(size=(border_width, width), layer="WG"))
    left.move((0, 0))
    right = c.add_ref(gf.components.rectangle(size=(border_width, width), layer="WG"))
    right.move((length - border_width, 0))

    # Entradas/Salidas
    n_ios = int(width // io_pitch)
    y0 = (width - (n_ios - 1) * io_pitch) / 2
    xs_io = gf.cross_section.strip(width=io_width, layer="WG")

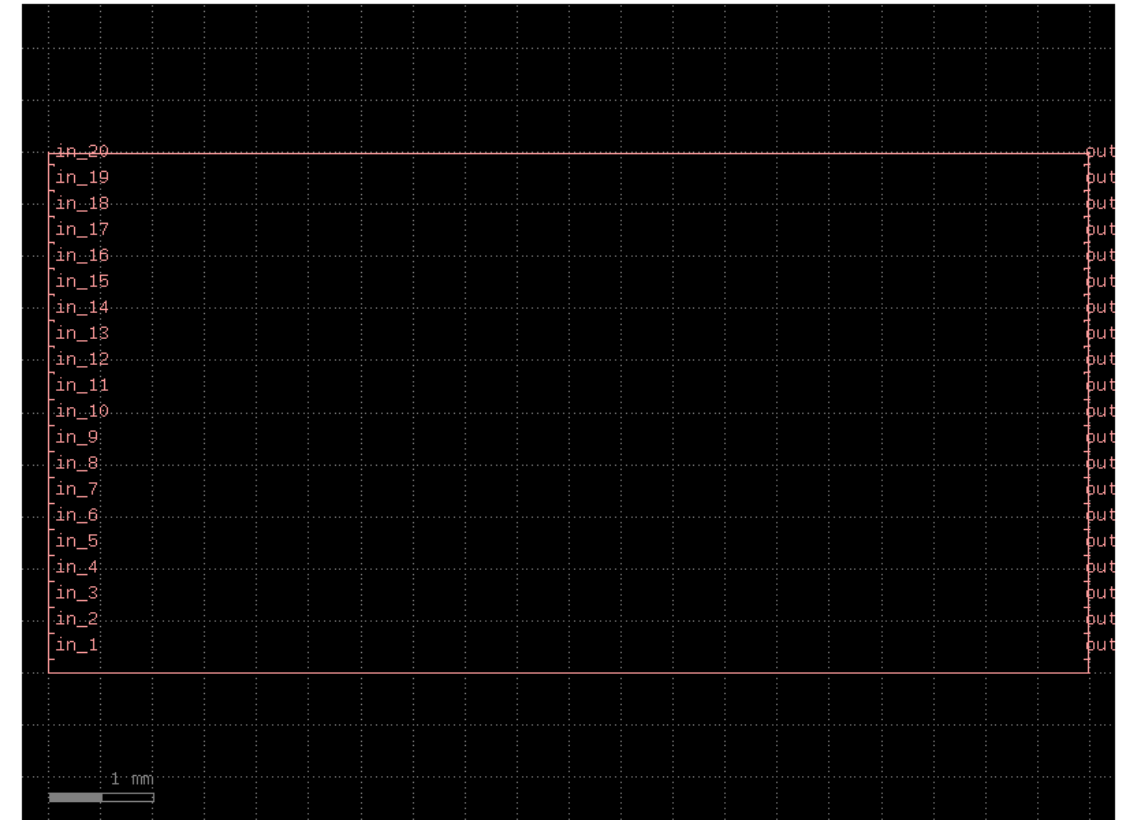
    for i in range(n_ios):
        y = y0 + i * io_pitch

        left_io = c.add_ref(gf.components.straight(length=io_length, cross_section=xs_io))
        left_io.move((0, y))

        right_io = c.add_ref(gf.components.straight(length=io_length, cross_section=xs_io))
        right_io.rotate(180)
        right_io.move((length, y))

        c.add_port(name=f"in_{i+1}", port=left_io.ports["o2"])
        c.add_port(name=f"out_{i+1}", port=right_io.ports["o2"])

    return c
```



- Create a cell component of this die, with I/Os accessible to connect

```
@gf.cell
def die_cell(
    width: float = 5000.0,
    length: float = 10000.0,
    border_width: float = 10.0,
    io_pitch: float = 250.0,
    io_width: float = 1.0,
    io_length: float = 50.0,
) -> gf.Component:

    c = gf.Component()

    # ----- Marco del die ----- #
    bottom = c.add_ref(gf.components.rectangle(size=(length, border_width), layer="WG"))
    bottom.move((0, 0))

    top = c.add_ref(gf.components.rectangle(size=(length, border_width), layer="WG"))
    top.move((0, width - border_width))

    left = c.add_ref(gf.components.rectangle(size=(border_width, width), layer="WG"))
    left.move((0, 0))

    right = c.add_ref(gf.components.rectangle(size=(border_width, width), layer="WG"))
    right.move((length - border_width, 0))

    # Entradas/Salidas
    n_ios = int(width // io_pitch)
    y0 = (width - (n_ios - 1) * io_pitch) / 2
    xs_io = gf.cross_section.strip(width=io_width, layer="WG")

    for i in range(n_ios):
        y = y0 + i * io_pitch

        left_io = c.add_ref(gf.components.straight(length=io_length, cross_section=xs_io))
        left_io.move((0, y))

        right_io = c.add_ref(gf.components.straight(length=io_length, cross_section=xs_io))
        right_io.rotate(180)
        right_io.move((length, y))

        c.add_port(name=f"in_{i+1}", port=left_io.ports["o2"])
        c.add_port(name=f"out_{i+1}", port=right_io.ports["o2"])

    return c
```



Part c) Floorplaning and die layout - Make an instance of the die as host component for your layout

```
gf.clear_cache()

main = gf.Component("main_layout")

die_ref = main.add_ref(
    die_cell(
        width=5000,
        length=10000,
        io_pitch=250,
        io_width=1.0,
        io_length=500,
        layer_box="FLOORPLAN",
    )
)

main.plot()
main.show()
```



- Add 3 sets of 3 straight waveguides, from left to right of the die, top, middle and bottom of the die

```
gf.clear_cache()

main = gf.Component("main_layout")

die_ref = main.add_ref(
    die_cell(
        width=5000,
        length=10000,
        io_pitch=250,
        io_width=1.0,
        io_length=500,
        layer_box="FLOORPLAN",
    )
)

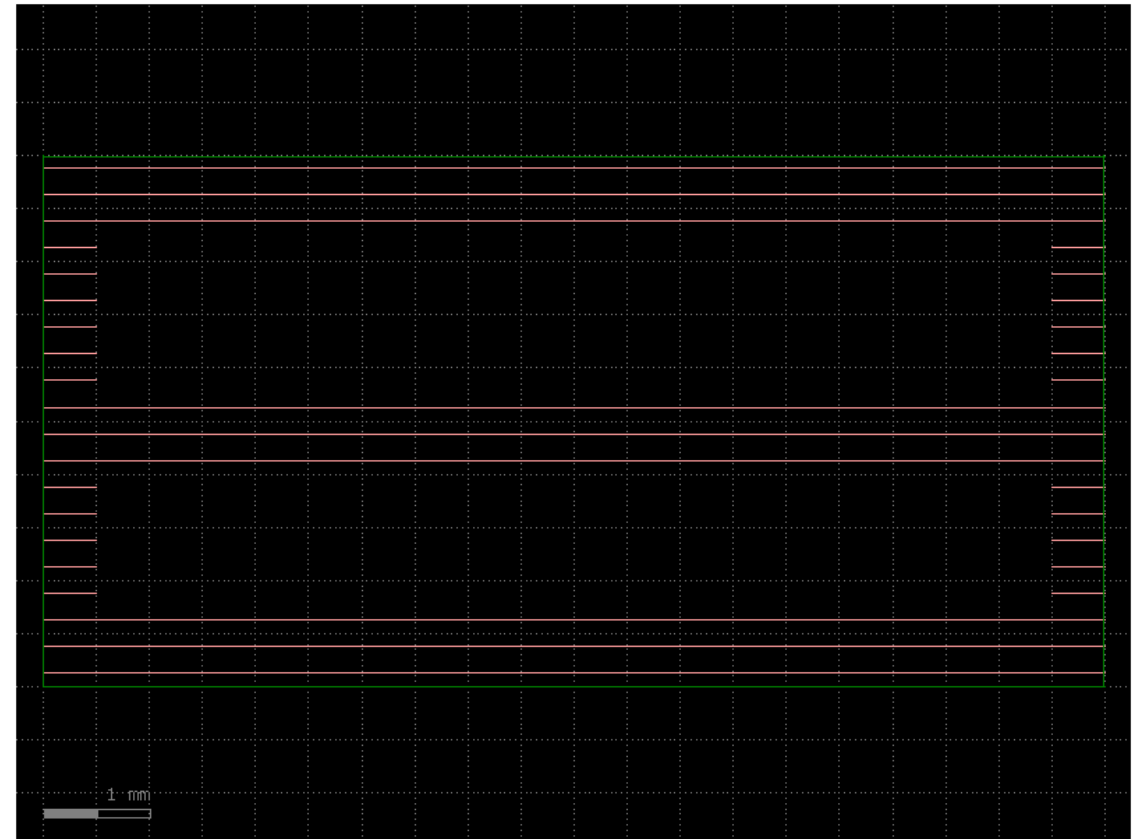
# Cross-section de las guías rectas
xs = gf.cross_section.strip(width=1.0, layer="WG")

# Índices de los I/Os que vamos a usar
bottom_ios = [1, 2, 3]
middle_ios = [9, 10, 11]
top_ios = [18, 19, 20]

straight_ios = bottom_ios + middle_ios + top_ios

# Añadimos las 9 guías rectas de izquierda a derecha
for i in straight_ios:
    gf.routing.route_single(
        main,
        port1=die_ref.ports[f"in_{i}"],
        port2=die_ref.ports[f"out_{i}"],
        cross_section=xs,
    )

main.plot()
main.show()
```



- Add 2 of each of the components above, with different lengths

```
device_refs = {}

# Posiciones dentro del die
x_left, x_right = 1500, 5500
y_positions = [3900, 3300, 2700, 2100, 1400] # MZI, MZI heater, Ring, Ring heater, Spiral

# 1) MZIs
mzi_1 = main.add_ref(mzi_unbalanced_2x2(delta_length=10, length_x=200))
mzi_1.dmovex(x_left).dmovey(y_positions[0])

mzi_2 = main.add_ref(mzi_unbalanced_2x2(delta_length=80, length_x=300))
mzi_2.dmovex(x_right).dmovey(y_positions[0])

device_refs["mzi_1"] = mzi_1
device_refs["mzi_2"] = mzi_2

# 2) MZIs con calentadores
mzi_h_1 = main.add_ref(mzi_unbalanced_2x2_heaters(delta_length=10, length_x=200))
mzi_h_1.dmovex(x_left).dmovey(y_positions[1])

mzi_h_2 = main.add_ref(mzi_unbalanced_2x2_heaters(delta_length=80, length_x=300))
mzi_h_2.dmovex(x_right).dmovey(y_positions[1])

device_refs["mzi_h_1"] = mzi_h_1
device_refs["mzi_h_2"] = mzi_h_2

# 3) Anillos all-pass
ring_1 = main.add_ref(all_pass_ring_mm12x2(extra_length=0))
ring_1.dmovex(x_left).dmovey(y_positions[2])

ring_2 = main.add_ref(all_pass_ring_mm12x2(extra_length=100))
ring_2.dmovex(x_right).dmovey(y_positions[2])

device_refs["ring_1"] = ring_1
device_refs["ring_2"] = ring_2

# 4) Anillos all-pass con calentador
ring_h_1 = main.add_ref(all_pass_ring_mm12x2_heater(extra_length=0))
ring_h_1.dmovex(x_left).dmovey(y_positions[3])

ring_h_2 = main.add_ref(all_pass_ring_mm12x2_heater(extra_length=100))
ring_h_2.dmovex(x_right).dmovey(y_positions[3])

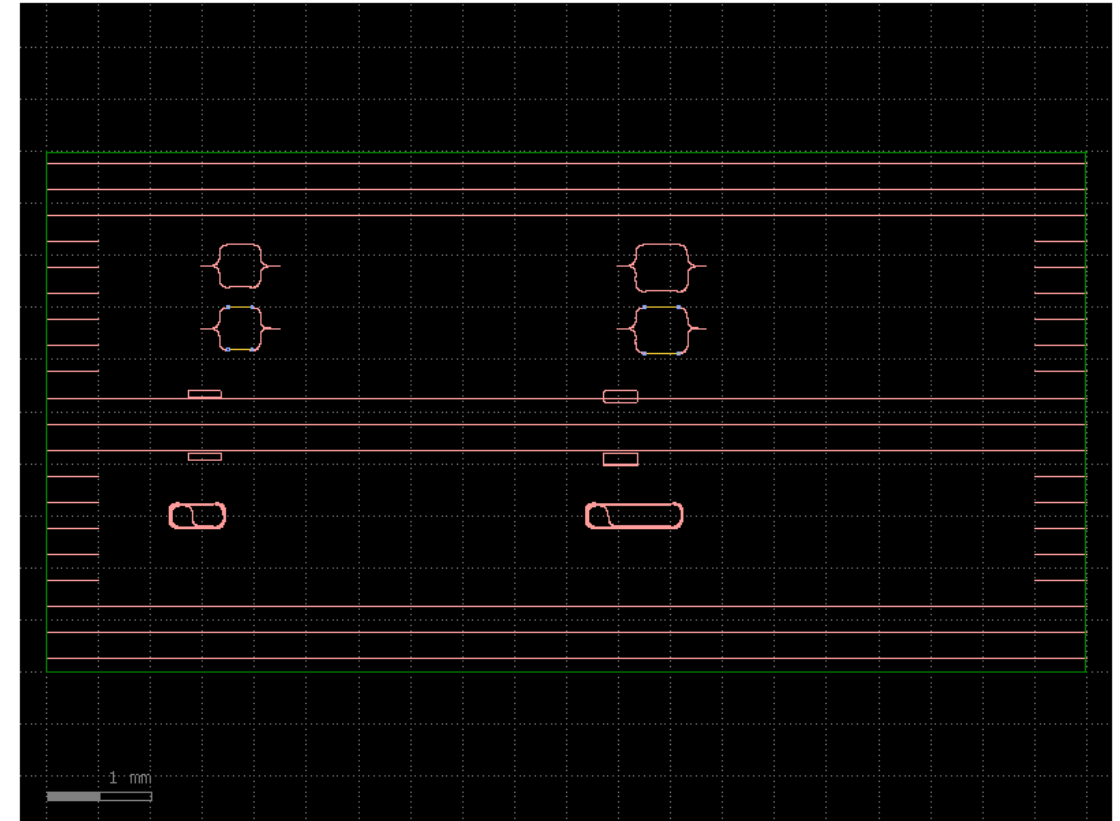
device_refs["ring_h_1"] = ring_h_1
device_refs["ring_h_2"] = ring_h_2

# 5) Espirales
spiral_1 = main.add_ref(spiral_length(length=100))
spiral_1.dmovex(x_left).dmovey(y_positions[4])

spiral_2 = main.add_ref(spiral_length(length=500))
spiral_2.dmovex(x_right).dmovey(y_positions[4])

device_refs["spiral_1"] = spiral_1
device_refs["spiral_2"] = spiral_2

main.plot()
main.show()
```



- Connect all your components to the I/Os

```
xs = "strip"

connections = [
    ("spiral_1", 4), ("spiral_2", 5),
    ("ring_h_1", 6), ("ring_h_2", 7),
    ("ring_1", 8), ("ring_2", 12),
    ("mzi_h_1", 13), ("mzi_h_2", 14),
    ("mzi_1", 15), ("mzi_2", 17),
]

for device_name, io_index in connections:
    dev = device_refs[device_name]

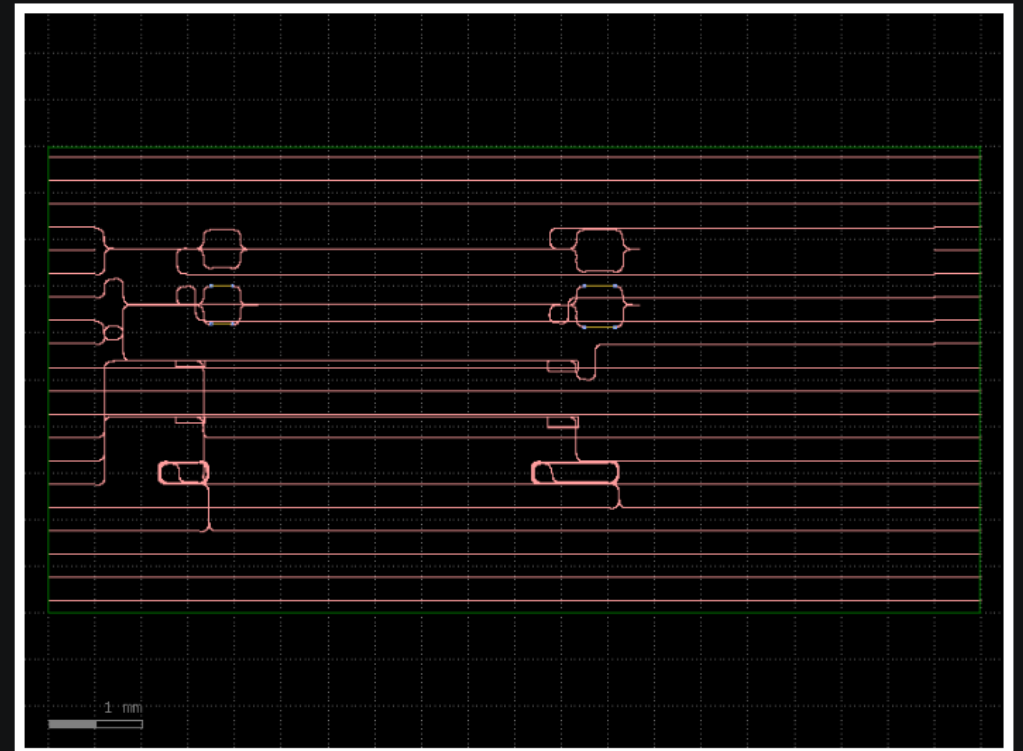
    print(f"Connecting {device_name}: in_{io_index} -> o1, o2 -> out_{io_index}")

    gf.routing.route_single(
        main,
        port1=die_ref.ports[f"in_{io_index}"],
        port2=dev.ports["o1"],
        cross_section=xs,
        allow_width_mismatch=True,
    )

    gf.routing.route_single(
        main,
        port1=dev.ports["o2"],
        port2=die_ref.ports[f"out_{io_index}"],
        cross_section=xs,
        allow_width_mismatch=True,
    )

main.plot()
main.show()
```

```
Connecting spiral_1: in_4 -> o1, o2 -> out_4
Connecting spiral_2: in_5 -> o1, o2 -> out_5
Connecting ring_h_1: in_6 -> o1, o2 -> out_6
Connecting ring_h_2: in_7 -> o1, o2 -> out_7
Connecting ring_1: in_8 -> o1, o2 -> out_8
Connecting ring_2: in_12 -> o1, o2 -> out_12
Connecting mzi_h_1: in_13 -> o1, o2 -> out_13
Connecting mzi_h_2: in_14 -> o1, o2 -> out_14
Connecting mzi_1: in_15 -> o1, o2 -> out_15
Connecting mzi_2: in_17 -> o1, o2 -> out_17
2026-06-12 09:48:26.779 | WARNING | kfactory.kcell:show:3958 - Could not connect to klive server
```





Thank you